

Configurable Multimodal Semantic Validation System (CMSVS)

Technical Report — Group 11 DSAI Lab Project

Indian Institute of Technology Madras — Deep Learning / Generative AI Course Project

Mallesh Mayara (21f2001118) · Mayank Dode (22f1000781) · Karthik Ganesh (21f2000775) · Ayush Verma (21f3000500)

Table of Contents

1. Executive Summary

2. Problem Statement & Motivation

3. Literature Review

4. Datasets

5. Preprocessing & Data Augmentation

6. System Architecture

7. Experiments & Model Selection

8. Evaluation Results

9. Error Analysis

10. Failed Experiments & Lessons Learned

11. Limitations & Future Work

12. Milestone Timeline

13. Team Contributions

14. References

1. Executive Summary

This report documents the full lifecycle of the **Configurable Multimodal Semantic Validation System (CMSVS)** — a domain-agnostic, zero-shot framework for extracting structured information from documents and validating consistency across heterogeneous document pairs.

Key results:

Metric	SBC Dataset	FUNSD Dataset
Overall F1 Score	0.7888	0.89 (2-class)
Precision	0.8200	0.91 (weighted)
Recall	0.7600	0.89 (weighted)
Accuracy	0.8944	0.8853
Total inference cost	\$0.00	\$0.00

The system achieves >85% cost reduction vs. Microsoft Azure Document Intelligence while requiring **zero labeled training data**.

2. Problem Statement & Motivation

2.1 The Problem

Enterprise document validation is labor-intensive and error-prone. Organizations routinely compare document pairs — Purchase Orders vs. Delivery Notes, Insurance Benefit Grids vs. SBC forms, contracts vs. amendments — to detect discrepancies. Existing solutions fail because:

- **Template-based systems** break on layout changes
- **Supervised ML approaches** require expensive labeled data (Azure Doc Intelligence: ~\$180/1K pages, 5+ labeled samples per template)
- **Text-diff tools** cannot recognize semantic equivalence ("Net 30" vs. "Payment due within 30 days")
- **LLM wrappers** lack output schema enforcement and evidence grounding

2.2 Our Solution

CMSVS decomposes document intelligence into two tasks:

1. **Config-Driven Custom NER** — Domain experts define entities in natural language YAML configs; MLLMs extract values zero-shot
2. **Chain-of-Thought Semantic Validation** — Extracted entity pairs are compared using structured CoT reasoning

2.3 Cost Comparison

Solution	Cost/1K pages	Training Data	Deployment Time
Microsoft Azure Document Intelligence	~\$180	5+ labeled samples	2–4 weeks
AWS Textract + Custom Model	~\$150–200	Labeled dataset	2–4 weeks
Manual Human Validation	\$500–2,000+	N/A	N/A
CMSVS (Ours)	~\$0	Zero	< 2 hours

3. Literature Review

3.1 Named Entity Recognition in Document Intelligence

Traditional NER evolved from CRF and BiLSTM-CRF to transformer-based models (BERT). Document-level NER models like LayoutLM, LayoutLMv2, and LayoutLMv3 incorporate 2D positional embeddings for spatial understanding but still require supervised fine-tuning on domain-specific data.

3.2 Vision-Language Models

Multimodal LLMs (GPT-4V, Claude 3, Gemini 1.5 Pro, LLaMA 4) accept image inputs directly, enabling layout-aware reasoning without OCR. Their instruction-following capabilities enable zero-shot extraction

with descriptive prompts — the core capability CMSVS exploits.

3.3 Semantic Validation

Semantic validation (NLI, STS tasks) determines meaning equivalence. Applying Chain-of-Thought prompting to pairwise entity-level validation across multimodal document pairs is novel — no prior work addresses this specific setting.

3.4 Gaps Addressed

Capability	Rule-Based	Fine-Tuned NER	Azure Doc Intel	CMSVS
Zero-shot extraction	×	×	×	✓
Custom entity types	Limited	With retraining	With labeled samples	NL config
Semantic validation	×	×	×	✓
Cross-domain w/o retraining	×	×	×	✓
Explainable decisions	×	×	×	✓
Cost per 1K pages	Low	Medium	~\$180	<\$25

4. Datasets

4.1 FUNSD — Form Understanding in Noisy Scanned Documents

Source: Public benchmark (Jaume et al., 2019)

Property	Value
Document type	Scanned administrative/business forms
Total form pairs evaluated	33
Total entity comparisons	279
Annotations	Bounding boxes, key-value pairs, linked entity groups
Noise characteristics	Scanner distortion, handwriting, stamps, low-contrast text
Entity types	Names, addresses, dates, numeric values, identifiers

Purpose: NER extraction benchmarking and validation accuracy on noisy scanned documents.

4.2 Healthcare SBC–Benefit Grid Dataset

Source: Purpose-built from publicly available Highmark PPO insurance plans

Property	Value
SBC Documents (Doc A)	20 unique documents across Gold/Silver/Bronze tiers
Benefit Grid Documents (Doc B)	20 paired documents (12 unaugmented, 8 augmented)

Property	Value
Ground Truth JSON files	8 annotated files with perturbation logs
Entities per config	18 across 5 sections
Total eligible entity comparisons	142
GT Matches	112
GT Mismatches	30

SBC Collection Challenges:

- Duplicate documents across retrieval paths required manual de-duplication via plan metadata inspection
- Formatting heterogeneity across carriers despite CMS-mandated template structure
- Multi-tier table structures with ambiguous cell-to-entity mapping

Benefit Grid template columns: Service Name, In-Network Cost, Out-of-Network Cost, Notes

5. Preprocessing & Data Augmentation

5.1 FUNSD Augmentation

Four augmentation categories were applied to test extraction robustness:

Category	Description	Example
Synonym Replacement	Semantically equivalent labels	"Interest Rate" → "Applicable Interest"
Numeric Format Variation	Different numeric representations	"11%" → "0.11"
Layout Modification	Structural changes (column reordering)	Column positions shuffled
OCR-Style Noise	Character-level substitutions	"0" vs "O", missing spaces

5.2 SBC Benefit Grid Augmentation

Distribution: 12 unaugmented (true MATCH) + 8 augmented (controlled MISMATCH/PARTIAL)

Category	Original → Modified	Expected Status
Semantic Changes	"No charge" → "Covered in full"	MATCH
Numeric Format	"\$6,550" → "6550"	MATCH (after normalization)
Value Conflicts	"\$525 copay" → "\$400 copay"	MISMATCH
Coverage Reclassification	"\$30 copay" → "Fully covered"	MISMATCH

5.3 Ground Truth Schema

Each ground truth JSON contains per-entity records with: entity name, values from both docs, normalized value, validation type, expected status, and perturbation log for augmented entities.

5.4 Validation Scenario Coverage

Scenario	Dataset Source
Exact Match	Unaugmented SBC pairs
Semantic Equivalence	Category 1 augmentation
Numeric Normalization	Category 2 augmentation
OCR Noise Handling	FUNSD augmentation
Conflict Detection	Category 3 augmentation
Coverage Change Detection	Category 4 augmentation

6. System Architecture

6.1 Design Principles

- 1. **Cost-First Design** — Free-tier APIs throughout; expensive LLM calls minimized via retrieval routing
- 2. **Clean Separation of Responsibilities** — Each component has exactly one job
- 3. **Configurable Without Code Changes** — Domain switching requires only a new YAML config
- 4. **Auditable by Design** — Every decision traceable to source document evidence

6.2 Seven-Layer Pipeline

Layer 1: Input Routing	– Detect PDF vs Image, route accordingly
Layer 2: OCR Processing	– PaddleOCR extracts text for indexing (PDF only)
Layer 3: Dense Vector Indexing	– NVIDIA NemoRetriever embeddings → ChromaDB
Layer 4: RAG Page Routing	– Cosine similarity retrieval of relevant pages
Layer 5: MLLM Extraction	– Visual entity extraction from page images
Layer 6: Expression Engine	– SimpleEval computes derived entity values
Layer 7: Section-Wise Validation	– CoT semantic comparison per section

6.3 Component Stack (Final)

Component	Tool	Provider	Cost
LLM Inference	llama-4-maverick-17b-128e-instruct	NVIDIA NIM (Free)	\$0
Dense Embeddings	llama-3.2-nemoretriever-300m-embed-v1	NVIDIA NIM (Free)	\$0

Component	Tool	Provider	Cost
Vector Store	ChromaDB (in-memory)	Local	\$0
OCR	PaddleOCR Mobile	Local	\$0
PDF Processing	PyMuPDF (fitz)	Local	\$0
Expression Evaluation	SimpleEval	Local	\$0

6.4 Two Processing Paths

PDF Path (Full RAG): PDF → OCR → Dense Index → Retrieve Pages → MLLM Extract → Expressions → Validate

Image Path (Direct): Image → Base64 Encode → MLLM Extract → Expressions → Validate

Both paths produce identical `FinalEntityValue` output structures.

6.5 Key Architectural Decisions

Why MLLM over OCR+NLP: Multi-column SBCs produce column-interleaved text under OCR; spatial context (which column a value belongs to) is lost in flat text. MLLM processes pages visually, preserving layout semantics.

Why Dense-Only Retrieval: The core challenge is semantic bridging — config descriptions use natural language while documents use domain terms. "The annual amount a member must pay before insurance begins covering costs" must route to pages containing "deductible." Dense embeddings capture this; keywords fail.

Why Section-Wise Processing: Batching related entities per section provides relational context, reduces API calls by ~72% (18 entities → 5 calls), and mirrors how domain experts review documents.

Why SimpleEval over eval(): Security — config files are authored by domain experts; `eval()` allows arbitrary code execution. SimpleEval enforces a strict operator whitelist.

6.6 Validation Engine

Fast Path (Rule-Based Pre-Normalization):

- Monetary: "\$1,500" → "1500.00 USD"
- Percentage: "20%" / "0.20" → "20.0%"
- Coverage: "No charge" / "Covered in full" → "0.00 USD"

Pairs normalizing to identical canonical forms are recorded as MATCH without LLM calls. **68.3% of entities resolved this way.**

Slow Path (CoT MLLM): Five-step reasoning: Normalization Review → Semantic Alignment → Discrepancy Analysis → Status Assignment → Confidence Calibration.

7. Experiments & Model Selection

7.1 Provider Evolution

Milestone	LLM Provider	Embedding Provider	Rationale
M3 (Design)	Groq (primary) + NVIDIA NIM (fallback)	NVIDIA NIM	Dual-provider for reliability
M4 (Implementation)	NVIDIA NIM only	NVIDIA NIM	Consolidated — single key, simpler code

Why the switch from Groq: NVIDIA NIM free tier provided sufficient throughput. Removing dual-provider logic eliminated inconsistency between runs, simplified codebase, and reduced configuration overhead.

7.2 Model Selection: llama-4-maverick-17b-128e-instruct

Criterion	Assessment
Multimodal capability	Native vision — accepts base64 page images directly
Context window	128K tokens — handles dense multi-column SBCs
JSON reliability	Consistent structured output with low schema violation rate
Instruction following	Accurate multi-step CoT validation sequences
Architecture	Mixture-of-Experts (128 experts)
Cost	Free on NVIDIA NIM

Models considered but not selected:

- llama-3.3-70b-versatile (Groq) — Strong but required separate provider
- mixtral-8x7b-32768 (Groq) — Good context window but lower extraction accuracy
- meta-llama/Llama-4-Scout-17B-16E (NVIDIA) — Evaluated as fallback; Maverick outperformed

7.3 Retrieval Top-K Ablation

K	Processing Time	F1	Accuracy	Notes
K=2	28.4s	0.742	0.871	Baseline
K=3	34.1s	0.789	0.893	+6.3% F1
K=4	41.7s	0.812	0.905	Best tradeoff (+9.4% F1)
K=5	51.2s	0.815	0.906	Marginal gain (+0.4%), 23% slower

Conclusion: K=4 recommended for production; K=3 for development.

7.4 Confidence Threshold Ablation

Threshold	Review Count	Precision	Recall	F1
-----------	--------------	-----------	--------	----

Threshold	Review Count	Precision	Recall	F1
0.60	1	0.730	0.784	0.756
0.70	2	0.782	0.776	0.779
0.75	3	0.820	0.761	0.789
0.80	5	0.851	0.710	0.774

Conclusion: 0.75 is balanced. For compliance-critical deployments, 0.70 maximizes mismatch detection (Recall).

7.5 Fast-Path vs. MLLM CoT Comparison

Path	Entity Count	% Total	F1	API Calls
Fast Path (Rule-Based)	97	68.3%	0.943	0
MLLM CoT	45	31.7%	0.722	45

Finding: The rule-based normalizer handles 68.3% of comparisons at zero API cost with F1=0.943, validating the fast-path design.

8. Evaluation Results

8.1 SBC Dataset — Aggregate Metrics

Metric	Value
Precision	0.8200
Recall	0.7600
F1 Score	0.7888
Accuracy	0.8944
True Positives	19
False Positives	4
True Negatives	108
False Negatives	6
Total Eligible	142

8.2 SBC — Per-Pair Results

Pair	Plan	F1	Errors
sbc_001	my Blue Access PPO Gold 0	1.000	0
sbc_002	my Blue Access PPO Silver 3700	1.000	0

Pair	Plan	F1	Errors
sbc_003	my Blue Access PPO Platinum 0	1.000	0
sbc_004	my Blue Access WV PPO Silver 700	1.000	0
sbc_005	State of Delaware Comprehensive PPO	0.727	3
sbc_006	NY State Employees HMO 210	0.667	2
sbc_007	PPO Blue	0.667	3
sbc_008	my Priority Blue Flex PPO Gold Premier	0.727	2

Pairs 001–004 (only synonym/OCR augmentations) achieved **perfect scores**. Pairs 005–008 (genuine conflict injections) are the challenging cases.

8.3 SBC — Per-Scenario Analysis

Scenario	Count	F1	Accuracy
exact_match	52	1.000	1.000
ocr_noise_injection	18	1.000	1.000
unit_swap	8	1.000	1.000
synonym_replacement	22	0.947	0.955
numeric_rounding_error	12	0.833	0.917
conflict_injection	20	0.722	0.800

8.4 SBC — Per-Section Analysis

Section	GT Mismatches	F1	Accuracy
Deductibles	4	1.000	1.000
Coverage Classifications	2	1.000	1.000
Out-of-Pocket Maximums	5	0.800	0.917
Copayments and Coinsurance	12	0.783	0.900
Prescription Drug Costs	7	0.714	0.875

8.5 SBC — Hardest vs. Easiest Entities

Hardest (Lowest F1):

Entity	F1	Root Cause
tier3_non_preferred_brand_copay	0.500	Multi-tier table ambiguity
individual_oop_max_out_of_network	0.571	Complex table structures

Entity	F1	Root Cause
urgent_care_copay	0.600	CoT reasoning errors

Easiest (Highest F1):

Entity	F1
individual_deductible_in_network	1.000
family_deductible_in_network	1.000
preventive_care_cost	1.000
mental_health_coverage	1.000

8.6 FUNSD Dataset — 2-Class Evaluation (Match vs. Conflict)

Metric	Value
Samples Evaluated	279
Accuracy	0.8853
Precision (Weighted)	0.91
Recall (Weighted)	0.89
F1 (Weighted)	0.89

Class	Precision	Recall	F1	Support
conflict	0.56	0.79	0.66	39
match	0.96	0.90	0.93	240

8.7 FUNSD Dataset — 3-Class Evaluation (Exact vs. Semantic vs. Conflict)

Metric	Value
Accuracy	0.7168
Precision (Weighted)	0.75
Recall (Weighted)	0.72
F1 (Weighted)	0.71

Class	Precision	Recall	F1	Support
conflict	0.56	0.79	0.66	39
exact_match	0.88	0.56	0.68	118
semantic_match	0.69	0.84	0.76	122

Key finding: The dominant error is confusion between exact and semantic matches — not missed conflicts. The system is reliable for production-level match vs. conflict decisions.

8.8 Token Consumption Profile

Stage	Tokens	Time
OCR Processing	0 (local)	3.2s
Embedding (Indexing)	4,800	2.1s
Embedding (Retrieval)	1,200	1.8s
MLLM Extraction	21,600	38.4s
MLLM Validation	18,700	35.2s
Total per pair	~46,300	~81s

9. Error Analysis

9.1 Error Summary (SBC Dataset)

Error Type	Count
False Negatives (missed mismatches)	6
False Positives (false alarms)	4
Total Errors	10

9.2 Error Categories

Category	Count	FN	FP	Primary Cause
COT_REASONING_ERROR	4	4	0	MLLM incorrect semantic judgment
NORMALIZATION_FAILURE	3	0	3	Normalizer missed equivalence
EXTRACTION_ERROR	2	2	0	Wrong value extracted from page
EXPRESSION_ERROR	1	0	1	Component variable extraction failed

9.3 Specific Error Cases

Pair	Entity	GT	System	Root Cause
sbc_005	urgent_care_copay	MISMATCH	MATCH	MLLM judged "No charge" vs "Coinsurance varies" as equivalent
sbc_005	rehabilitation_services	MISMATCH	MATCH	15% vs 20% coinsurance not flagged

Pair	Entity	GT	System	Root Cause
sbc_006	specialist_copay	MISMATCH	MATCH	Extracted wrong tier from multi-column table
sbc_007	tier1_generic_copay	MISMATCH	MATCH	"20% with \$10 min/\$100 max" vs "20%" treated as equivalent
sbc_007	preventive_care	MATCH	MISMATCH	"No charge at all" not in normalizer dictionary
sbc_007	physician_surgeon_fees	MATCH	MISMATCH	"10 %" (space artifact) not normalized
sbc_008	hospital_stay_facility_fee	MISMATCH	MATCH	Extracted \$400 instead of \$525 from dense table
sbc_008	tier2_preferred_brand	MATCH	MISMATCH	EXPRESSION variable extraction failed

9.4 Key Error Patterns

1. **Guardrail omission (2 FN):** MLLM fails to detect when Doc B omits cost guardrails (min/max limits) present in Doc A

2. **Three-tier column ambiguity (2 errors):** Multi-column SBC tables cause extraction from wrong column

3. **Normalizer gap (3 FP):** Missing phrasings like "No charge at all" and space artifacts like "10 %"

10. Failed Experiments & Lessons Learned

10.1 Groq as Primary Provider (Abandoned in M4)

What we tried: Groq's LPU architecture (500–800 tokens/sec) as primary LLM with NVIDIA NIM as fallback.

Why it failed: Dual-provider logic introduced inconsistent behavior between runs. Provider-switching added code complexity. NVIDIA NIM alone provided sufficient throughput.

Lesson: Single-provider simplicity outweighs marginal speed advantages when free-tier quotas are sufficient.

10.2 Per-Entity Extraction (Replaced by Section-Batching)

What we tried: Extracting one entity at a time with focused prompts (M1 design).

Why it failed: 18 API calls per document instead of 5. No relational context between related entities (e.g., individual vs. family deductible).

Lesson: Section-batching provides 72% API call reduction and better extraction accuracy through contextual grounding.

10.3 Text-Based Extraction over OCR Output

What we tried: Standard PDF text extraction for entity values.

Why it failed: Multi-column SBC PDFs produced column-interleaved text. Spatial relationships between headers and cell values were lost. Table structures were not reconstructable.

Lesson: Visual MLLM extraction is strictly superior for structured documents. OCR is sufficient only for building the retrieval index.

10.4 Keyword-Based (BM25) Retrieval

What we tried (considered in M3 design): Hybrid dense + keyword retrieval.

Why it was rejected: Config descriptions use natural language ("annual amount a member must pay before insurance coverage begins") while documents use domain terms ("deductible"). Keyword matching fails entirely for this semantic gap. Dense-only retrieval handles it naturally.

10.5 Top-K=2 Retrieval (Underperformed)

What we tried: Default K=2 pages per section.

Result: F1=0.742 — lowest in ablation. Entities spanning pages or in unexpected locations were missed.

What we picked: K=4 (F1=0.812, +9.4% improvement) as the best accuracy/latency tradeoff.

11. Limitations & Future Work

11.1 Current Limitations

Limitation	Impact	Component
Three-tier SBC table column ambiguity	Medium	MLLM Extraction
EXPRESSION entities propagate upstream errors	Medium	Expression Engine
ValueNormalizer incomplete coverage	Low-Medium	Fast-path validation
NVIDIA NIM rate limits (~3–5s delays)	Low	All API stages
Guardrail clauses not treated as distinct attributes	Medium	CoT Validation

11.2 Proposed Improvements

Priority	Improvement	Expected Gain
High	Expand normalizer with 15+ new phrasings	+3 FP eliminated
High	Guardrail-aware CoT prompt (flag omitted min/max)	+2 FN recovered
High	Three-tier column disambiguation in extraction prompt	+2 FN recovered
Medium	Cross-verification pass for high-stakes entities	Reduces FN risk

Priority	Improvement	Expected Gain
Medium	Numeric range validator for EXPRESSION entities	Prevents silent failures
Low	Increase default top-K from 3 to 4	+2.9% F1

12. Milestone Timeline

Milestone	Title	Key Deliverables
M1	Problem Definition & Literature Review	Problem formulation, gap analysis, competitive comparison
M2	Dataset Preparation & Preprocessing	FUNSD benchmark adaptation, 20 SBC–Benefit Grid pairs, 4-category augmentation, 8 GT JSON files
M3	Model Architecture	7-layer pipeline design, component selection, RAG routing, section-wise validation, YAML config schema
M4	Model Training & Experiments	Working E2E solution, NVIDIA NIM consolidation, Streamlit demo, initial parameter selection
M5	Model Evaluation & Analysis	Formal metrics (P/R/F1), ablation experiments, 10-error root cause analysis, per-scenario benchmarks
M6	Deployment & Documentation	FastAPI + Streamlit deployment, Hugging Face Spaces, user guide, non-technical report

13. Team Contributions

Member	Key Contributions
Mallesh Mayara (21f2001118)	Config architecture, shared type contracts, report generator, semantic validation engine, pipeline integration (PDF/Image/Master), Streamlit UI, end-to-end testing, all milestone documentation
Mayank Dode (22f1000781)	Input routing, PDF rendering, OCR integration, FUNSD augmentation pipeline, batch pipeline runner for evaluation
Karthik Ganesh (21f2000775)	Dense retrieval architecture (IndexBuilder, DenseRetriever), NVIDIA NIM client, provider evaluation, entity name mapper, evaluation metrics
Ayush Verma (21f3000500)	NER prompt engineering, MLLM extractor, expression evaluator/orchestrator, SBC augmentation, GT JSON generation, ablation experiments

14. References

1. Jaume et al., "FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents," ICDAR 2019

2. Xu et al., "LayoutLM: Pre-training of Text and Layout for Document Image Understanding," KDD 2020

3. Microsoft Azure Document Intelligence documentation
4. NVIDIA NIM API documentation
5. PaddlePaddle, "PaddleOCR: Awesome multilingual OCR toolkit"
6. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS 2022